App.py

```
from flask import Flask, render_template, request, redirect, url_for, send_from_directory
import mysql.connector
import os
from werkzeug.utils import secure_filename
import PyPDF2
import pyttsx3
from groq import Groq
from pptx import Presentation

app = Flask(__name__)

UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

# ---------------- DATABASE ----------------
db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",   # XAMPP default
    database="interactive_learning_system"
)
cursor = db.cursor(dictionary=True)

# ---------------- HELPER FUNCTIONS ----------------

def structure_text(raw_text):
    lines = raw_text.split("\n")
    structured = []
```

```
    for line in lines:
        line = line.strip()
        if not line:
            continue
        if all(char in "-_=*•." for char in line):
            continue
        structured.append("• " + line)

    return "\n".join(structured)

def chunk_text(text, max_lines=10):
    lines = text.split("\n")
    chunks = []
    current = []

    for line in lines:
        if line.strip():
            current.append(line)
        if len(current) == max_lines:
            chunks.append("\n".join(current))
            current = []

    if current:
        chunks.append("\n".join(current))

    return chunks

def generate_explanation_with_ai(chunks):
    api_key = os.environ.get("GROQ_API_KEY")
    explanations = []
```

```
    # 🔐 SAFETY FALLBACK
    if not api_key:
        return "\n".join(chunks)

    client = Groq(api_key=api_key)

    for chunk in chunks:
        prompt = f"""
You are a helpful teacher.
Explain the following content in simple, clear, student-friendly language.
Do not add new concepts. Only explain what is given.

Content:
{chunk}
"""

        try:
            response = client.chat.completions.create(
                model="llama-3.1-8b-instant",
                messages=[
                    {"role": "user", "content": prompt}
                ]
            )
            explanations.append(response.choices[0].message.content)

        except Exception:
            explanations.append(chunk)

    return "\n\n".join(explanations)

def group_into_slides(text, points_per_slide=5):
```

```
    lines = text.split("\n")
    slides = []
    current_slide = []

    for line in lines:
        if line.strip():
            current_slide.append(line)

        if len(current_slide) == points_per_slide:
            slides.append(current_slide)
            current_slide = []

    if current_slide:
        slides.append(current_slide)

    return slides

def separate_slide_and_narration(slides, max_visible_points=2):
    separated = []

    for slide in slides:
        slide_text = slide[:max_visible_points]   # condensed
        narration_text = slide                    # full explanation

        separated.append({
            "slide_text": slide_text,
            "narration_text": narration_text
        })

    return separated
```

```
def generate_voice_for_slide(text, output_path):
    engine = pyttsx3.init()
    engine.setProperty("rate", 150)   # speaking speed
    engine.save_to_file(text, output_path)
    engine.runAndWait()

def extract_slides_from_ppt(ppt_path):
    prs = Presentation(ppt_path)
    slides_content = []

    for slide in prs.slides:
        slide_text = []

        for shape in slide.shapes:
            if hasattr(shape, "text"):
                text = shape.text.strip()
                if text:
                    slide_text.append(text)

        if slide_text:
            slides_content.append(slide_text)

    return slides_content
def chunk_large_text(text, max_chars=3000):
    chunks = []
    current = ""

    for line in text.split("\n"):
        if len(current) + len(line) < max_chars:
            current += line + "\n"
        else:
```

```
        chunks.append(current)
        current = line + "\n"

    if current:
        chunks.append(current)

    return chunks

def generate_slide_structure_with_ai(text):
    api_key = os.environ.get("GROQ_API_KEY")
    if not api_key:
        return ""

    client = Groq(api_key=api_key)

    prompt = f"""
Extract ONLY keywords and short phrases for presentation slides.

STRICT RULES:
- NO sentences
- NO explanations
- NO verbs like explain, means, describes
- EACH bullet MAX 3–5 words
- Think like slide bullets, not notes

FORMAT STRICTLY:

Slide 1:
Title: <topic>
Points:
- keyword
```

```
- keyword
- keyword

Slide 2:
Title: <topic>
Points:
- keyword
- keyword

CONTENT:
{text}
"""

    response = client.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=[{"role": "user", "content": prompt}]
    )

    return response.choices[0].message.content

def parse_slide_structure(ai_text):
    slides = []
    current = None

    for line in ai_text.split("\n"):
        line = line.strip()

        # Start a new slide
        if line.lower().startswith("slide"):
            if current:
                slides.append(current)
```

```
            current = {"title": "Untitled Slide", "points": []}

        # Slide title
        elif line.lower().startswith("title:"):
            if current:
                current["title"] = line.replace("Title:", "").strip()

        # Bullet point
        elif line.startswith("-"):
            if current is None:
                # Create a slide if GPT forgot to start one
                current = {"title": "Auto Generated", "points": []}
            current["points"].append(line.replace("-", "").strip())

    if current:
        slides.append(current)

    return slides




def create_ppt_from_slides(slides, output_path):
    prs = Presentation()

    for slide_data in slides:
        slide = prs.slides.add_slide(prs.slide_layouts[1])
        slide.shapes.title.text = slide_data["title"]

        body = slide.shapes.placeholders[1].text_frame
        body.clear()
```

```
        for point in slide_data["points"]:
            body.add_paragraph().text = point
            body.add_paragraph().level = 1

    prs.save(output_path)

# ---------------- ROUTES ----------------

@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        role = request.form["role"]
        email = request.form["email"]
        password = request.form["password"]

        query = "SELECT * FROM users WHERE email=%s AND password=%s AND role=%s"
        cursor.execute(query, (email, password, role))
        user = cursor.fetchone()

        if user:
            if role == "teacher":
                return redirect(url_for("teacher_dashboard"))
            elif role == "student":
                return redirect(url_for("student_dashboard"))
            elif role == "parent":
                return redirect(url_for("parent_dashboard"))

    return render_template("login.html")

@app.route("/upload_notes", methods=["POST"])
def upload_notes():
```

```
    if "notes_file" not in request.files:
        return "No file uploaded"

    file = request.files["notes_file"]
    if file.filename == "":
        return "No selected file"

    filename = secure_filename(file.filename)
    file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))

    return redirect(url_for("teacher_dashboard"))

@app.route("/uploads/<filename>")
def uploaded_file(filename):
    return send_from_directory(app.config['UPLOAD_FOLDER'], filename)

@app.route("/teacher")
def teacher_dashboard():
    files = [
    f for f in os.listdir(app.config['UPLOAD_FOLDER'])
    if f.lower().endswith((".pdf", ".ppt", ".pptx"))
    and not f.endswith("_auto.pptx")
]

    return render_template("teacher_dashboard.html", files=files)

@app.route("/student")
def student_dashboard():
    return render_template("student_dashboard.html")

@app.route("/parent")
```

```
def parent_dashboard():
    return render_template("parent_dashboard.html")

@app.route("/extract_text/<filename>")
def extract_text(filename):
    file_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)

    # ======================================================
    # CASE 1: PPT / PPTX uploaded → extract slides directly
    # ======================================================
    if filename.lower().endswith((".ppt", ".pptx")):
        slides_raw = extract_slides_from_ppt(file_path)

        slides = []
        for slide in slides_raw:
            slide_text = slide  # PPT slide content stays as-is

            narration_prompt = [
                "Explain the following slide content in detail for students:\n" +
                "\n".join(slide)
            ]

            narration = generate_explanation_with_ai(narration_prompt)

            slides.append({
                "slide_text": slide_text,                # visible bullets
                "narration_text": narration.split("\n")     # expanded explanation
            })

        return render_template(
            "slides_view.html",
```

```
        slides=slides,
        filename=filename
    )

# ======================================================
# CASE 2: PDF uploaded → auto-generate PPT first
# ======================================================
elif filename.lower().endswith(".pdf"):
    raw_text = ""

    with open(file_path, "rb") as pdf_file:
        reader = PyPDF2.PdfReader(pdf_file)
        for page in reader.pages:
            if page.extract_text():
                raw_text += page.extract_text() + "\n"

    # 🔹 Chunk PDF text safely
    text_chunks = chunk_large_text(raw_text)

    all_slide_data = []

    for chunk in text_chunks:
        ai_structure = generate_slide_structure_with_ai(chunk)
        slide_data = parse_slide_structure(ai_structure)
        all_slide_data.extend(slide_data)

    # 🔹 Remove empty slides
    all_slide_data = [
        s for s in all_slide_data
        if s.get("points")
    ]
```

```
    # 🔹 Create PPT (export)
    ppt_name = filename.replace(".pdf", "_auto.pptx")
    ppt_path = os.path.join(app.config["UPLOAD_FOLDER"], ppt_name)
    create_ppt_from_slides(all_slide_data, ppt_path)

    # 🔹 Build slide + narration (THIS is what fixes your main issue)
    slides = []

    for slide in all_slide_data:
        slide_points = slide["points"]   # SHORT bullets

        narration = generate_explanation_with_ai([
            "Explain these points clearly for students:\n" +
            "\n".join(slide_points)
        ])

        slides.append({
            "slide_text": slide_points,          # ✅ bullets only
            "narration_text": narration.split("\n")  # ✅ explanation
        })

    return render_template(
        "slides_view.html",
        slides=slides,
        filename=ppt_name
    )


# ==================================================
# CASE 3: Unsupported file
```

```
    # ====================================================
    else:
        return "Unsupported file format"



@app.route("/test_voice")
def test_voice():
    sample_text = "Hello. This is a test voice generated for the learning management system."
    output_path = "static/test_voice.mp3"
    generate_voice(sample_text, output_path)
    return "Voice generated successfully"

@app.route("/generate_voice/<filename>", methods=["POST"])
def generate_voice_route(filename):
    slides = request.form

    audio_folder = os.path.join("static", "audio", filename)
    os.makedirs(audio_folder, exist_ok=True)

    slide_number = 1

    for key in slides:
        if key.startswith("narration_"):
            narration_text = slides[key]

            audio_path = os.path.join(
                audio_folder,
                f"slide_{slide_number}.mp3"
            )

            generate_voice_for_slide(narration_text, audio_path)
```

```
        slide_number += 1

    return "Voice generated successfully"

@app.route("/create_video/<filename>", methods=["POST"])
def create_video(filename):
    # TEMPORARY PLACEHOLDER
    return f"Video creation started for {filename}. (Next step)"

# ---------------- RUN ----------------
if __name__ == "__main__":
    app.run(debug=True)
```

groq_test.py

```
import os
from groq import Groq

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

prompt = """
Explain the concept of Fibonacci sequence in simple terms for a student.
"""

response = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {"role": "user", "content": prompt}
    ]
)
```

```
print("AI Response:\n")
print(response.choices[0].message.content)
```

slides_view.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Slide-wise Content</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard.css') }}">
</head>
<body>

<div class="wrapper">

  <div class="sidebar">
    <h2>LMS</h2>
    <a href="/teacher">← Back to Dashboard</a>
  </div>

  <div class="main">

    <div class="header">
      <h2>Slide-wise Explanation</h2>
      <p>File: {{ filename }}</p>
    </div>

<form method="POST" action="{{ url_for('create_video', filename=filename) }}">
      {% for slide in slides %}
      <div class="card">
        <h3>Slide {{ loop.index }}</h3>
```

```
            <p><strong>Slide Content (Visible to students)</strong></p>
            <p><strong>Slide Content (Editable)</strong></p>
            <textarea name="narration_{{ loop.index }}"
        style="width:100%; height:150px;">
          {% for line in slide.narration_text %}
          {{ line }}
          {% endfor %}
          </textarea>


            <p><strong>Narration (Stickman speech)</strong></p>
            <textarea style="width:100%; height:150px;">
          {% for line in slide.narration_text %}
          {{ line }}
          {% endfor %}
            </textarea>
          </div>
          {% endfor %}

          <button type="submit" class="btn">
  Create Video
</button>

</form>

    </div>
</div>

</body>
</html>
```

Structured_text.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Structured Content</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard.css') }}">
</head>
<body>

<div class="wrapper">

  <div class="sidebar">
    <h2>LMS</h2>
    <a href="/teacher">← Back to Dashboard</a>
  </div>

  <div class="main">

    <div class="header">
      <h2>Structured Explanation</h2>
      <p>File: {{ filename }}</p>
    </div>

    <div class="card">
      <h3>Edit Content (Teacher can modify)</h3>

      <form method="POST">
        <textarea style="width:100%; height:400px; padding:10px;">
{{ text }}
        </textarea>
```

```
            <br><br>
            <button disabled>Save & Generate Video (Next Phase)</button>
          </form>
        </div>

    </div>
  </div>

</body>
</html>
```